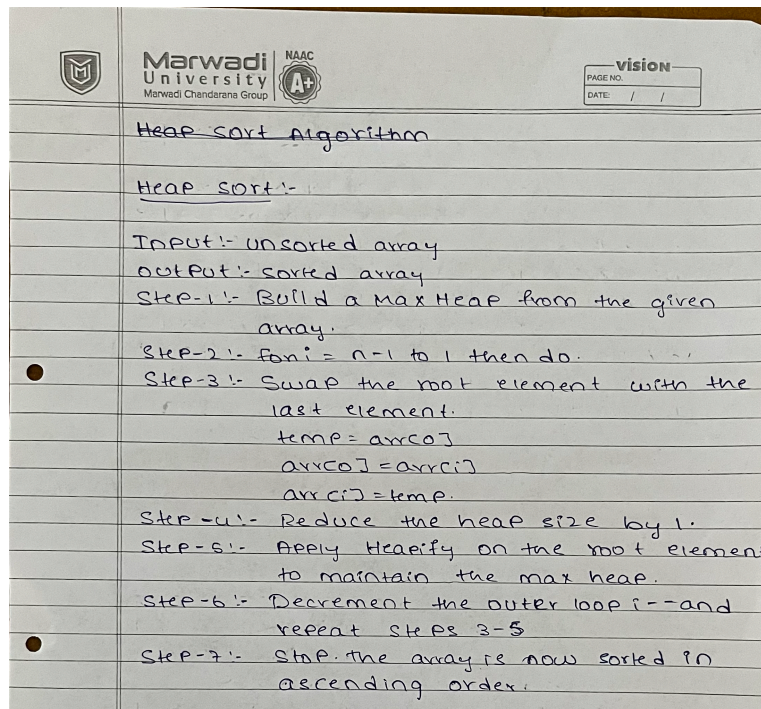


PRACTICAL - 3

AIM : Implementation of Max - heap sort algorithm.

ALGORITHM :



PROGRAM :

```
#include <iostream>
using namespace std;

void heapify(int arr[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i)
    {
        swap(arr[i], arr[largest]);
    }
}
```

```
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n)
{
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--)
    {
        swap(arr[0], arr[i]);

        heapify(arr, i, 0);
    }
}

int main()
{
    int n;

    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];

    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }

    heapSort(arr, n);

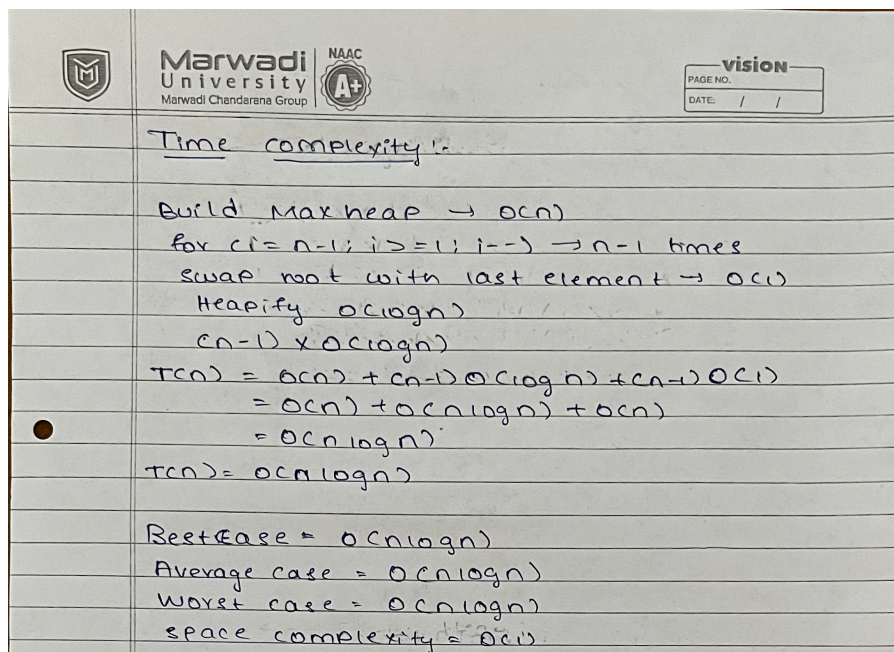
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }

    return 0;
}
```

OUTPUT :

```
SYNC STORAGE/UNIVERSITY STUFF/DAA_LAB on ↗ master
● > ./HeapSort
Enter number of elements: 4
Enter 4 elements: 09 11 07 30
Sorted array: 7 9 11 30 %
```

TIME COMPLEXITY :



CONCLUSION : I successfully implemented the Heap Sort algorithm and understood how it works using a Max Heap. I learned how the largest element is repeatedly placed at the end of the array and how the heap is maintained using the heapify operation. From the time complexity analysis, I understood that Heap Sort takes $O(n \log n)$ time in the best, average, and worst cases. I also learned that it is an in-place sorting algorithm with $O(1)$ auxiliary space complexity.